SI 650 Project Proposal: Brainstorming Chat Log

2026-10-11 · Conversation between the student and Claude (Claude Code), used for brainstorming the SI 650 / EECS 549 project proposal. Replies are lightly condensed.

Turn 1: Reading the proposal PDF

> **You:** read the pdf file and tell me what do we need to do for the project? also, what is the requirement? any ir related project is acceptable? netflix searching thing is applicable?

Claude: (read SI650___Project_Proposal.pdf)

What you need to do. The proposal is a PDF with six graded sections, 100 points in total:

Section	Points	What it needs
Introduction	10	The problem, the broader context, and why it matters (who cares, and what changes if you solve it). Write it last.
IR Task Definition	20	The specific IR task, with input and output. Give example queries and example documents. If the system is queryless, state the implicit information need.
Data	20	The source, how you collect it, preprocessing, a few examples, and a rough statistics table. Say what the queries are and how relevance is defined and where the labels come from (self-annotation is fine). Name a backup data source.
Related Work	10	At least 5 related papers, each with a few sentences on what they did. Say how your problem or approach differs.
Evaluation and Results	30	A specific quantitative metric, such as nDCG, MAP or P@k. You need a random baseline and a simple baseline like untuned BM25.
Work Plan	10	What is done so far and what remains, as weekly SMART milestones (specific, measurable, concrete).

What to submit

1. A PDF of the proposal.
2. Your genAI prompts and chat history. This is required even if you used no genAI, in which case you submit a short document saying so.

Policies to note

- GenAI use: You may use it to ideate and polish, and to ask clarifying or conceptual questions. You may not use it to generate the proposal or parts of it.
- Late days: You get 5 free late days for the whole semester. Any minute past the deadline costs a full day.

- Deadline: The PDF says "see Canvas".
- After the proposal: The Week 11 update needs all your data in hand plus a simple baseline. The Week 14 update is a presentation.

Is any IR project acceptable? Is a Netflix-style search OK? The PDF doesn't give a list of approved topics. It says "one problem or topic of their choice", so nearly any IR task works if it can be defined concretely. Netflix-style search is a valid fit.

An example framing: given a natural-language query ("a feel-good sci-fi movie with a twist ending"), retrieve and rank movies or shows. Queries are text, documents are title records (description, genre, cast), and relevance is judged per query-title pair.

Things to watch for:

- You need real data and relevance labels. The PDF discourages data that needs permissions. Netflix's internal logs aren't available, so use public sources like the Kaggle Netflix titles dataset, MovieLens, TMDB or IMDb, and name a backup. You must define queries and relevance judgments yourself, for example by annotating a set of queries.
- Measurability is graded most heavily (30 points). "Find movies similar to X" or recommendation is allowed (queryless systems are mentioned), but you must state the implicit information need and a measurable metric.
- Avoid pure topic modeling or clustering. The PDF warns these are very hard to evaluate unless tied to a downstream task.
- Make it more than a keyword search. BM25 is your simple baseline, so the project should show something beyond it, such as dense retrieval, query expansion, or LLM-based reranking.

Turn 2: IR applications and project ideas

> **You:** give me some ideas of IR. like i dont quite really understand the ir application in real life. i know google search, i know youtube search, netflix search, what else? also, any novel idea?

Claude:

IR is any system that takes an information need (a query, or an implicit need) and returns a ranked list of relevant items out of a large collection. Search engines are the best-known case, but the same idea appears in many other places.

IR in real life, beyond Google, YouTube and Netflix

Domain	Example	Query to documents
Legal	Westlaw, e-discovery	"cases about landlord liability for mold" to court opinions
Medical	PubMed, clinical trial matching	patient description to eligible clinical trials

Code	GitHub search, Stack Overflow	"parse JSON in Go" to code snippets or answers
E-commerce	Amazon, Etsy	"gift for a hiker dad" to products
Enterprise	Slack, Notion, email search	"Q3 budget doc from Sam" to messages and files
Academic	Google Scholar, Semantic Scholar	topic or abstract to papers
Recipes / food	Allrecipes, Yummly	"vegan, 30 mins, I have lentils" to recipes
Music	Spotify, Shazam	hummed tune or mood to songs
Maps / local	Google Maps, Yelp	"quiet cafe to study near me" to places
Jobs	LinkedIn	resume to job postings (and the reverse)
QA / RAG	ChatGPT with retrieval, customer-support bots	question to passages fed to an LLM
Images	Pinterest, Google Lens	photo to similar products
Conversational	Voice assistants	spoken question to answer or action

Many are "queryless" (recommendations, feeds). The implicit need there is "what would this user want next?".

Project ideas. Each is scoped so you can get data and measure it within a semester.

Fairly standard but solid:

1. Natural-language movie/show search. Vague, mood-based queries ("something cozy like Ghibli but live action") against titles. Compare BM25, dense embeddings and an LLM reranker.
2. Recipe search by constraints. The query mixes ingredients, dietary needs and time. Hard for keyword search because of negation ("without nuts").
3. Course search for a university. Retrieve courses from vague goals ("learn to build recommender systems, no heavy math"). Course catalogs are public and you can annotate relevance yourself.

More novel angles:

4. Search for queries with negation or exclusion. Embedding models are known to handle "not X" poorly. Build a small benchmark in one domain and test whether rerankers fix it.
5. Tip-of-the-tongue retrieval. People describe things they can't name. Reddit's r/tipofmytongue gives real queries, and the accepted answers serve as built-in relevance labels.
6. Fact-checking retrieval. Given a claim from social media, retrieve the evidence passages that support or refute it.
7. Privacy-policy or terms-of-service search. "Can this app sell my location data?" to the relevant clause.
8. Search over your own data. Personal notes, lecture slides, or course Piazza posts. You write the queries and judge relevance yourself, which cuts the data risk.
9. Explainable retrieval. Return results plus a generated reason why each is relevant, and measure whether the explanation helps users judge relevance.

10. Cross-lingual or code-switched search. Query in one language and retrieve in another, or mix languages in the query.

How to choose. Judge each idea by three questions, which match the grading rubric:

- Can I get the documents without asking for permission?
- Can I define relevance and get labels, either public ones (like the accepted answers in #5) or ones I annotate myself, say 30 to 50 queries?
- Is there a gap beyond BM25 that makes the project interesting?

Ideas 4, 5 and 7 are the strongest on the "novel and measurable" front. Idea 5 has the best built-in labels, and idea 8 has the lowest data risk.

Turn 3: Movie search by genre, lines and character name

> **You:** im thinking if someone search by by genre, lines or maybe character name, the output should give the exact movie or movies that match the most to the query. does that count novel?

Claude:

Not as stated, but it's a solid project, and novelty isn't graded anyway. The rubric scores a clear task, data, a metric and baselines, not originality.

Assessment

- Genre alone isn't a good query for finding "the exact movie". Thousands of movies share a genre, so it works as a filter or facet, not a main signal.
- Character name is easy. It's basically an exact-match lookup, and BM25 or a database query already solves it. There's little gap to show.
- Lines (quotes) are the interesting part. People rarely remember the exact wording ("Luke, I am your father" is actually "No, I am your father"). Retrieving a movie from a partial, paraphrased or misremembered line is hard for keyword search, and that's where BM25 and dense retrieval differ.

So the quote part carries the project, and genre and character name become supporting signals.

Ways to sharpen it

1. Misremembered or paraphrased quote to movie. Compare exact-match BM25, dense embeddings and a hybrid. This is the cleanest "beats the baseline" story.
2. Mixed-signal queries. "Sci-fi movie, a robot named Wall-E says 'directive'..." The task is to combine fields (genre, character, dialogue, plot) and test which signals matter most.
3. Tip-of-the-tongue style. Add scene descriptions. Real queries exist on Reddit's r/tipofmytongue.

Why this fits the rubric well

- Relevance is nearly free. Each query has one known target movie. This is "known-item search", so you evaluate with MRR or Success@k without hand-labeling every document.
- Data is public: Cornell Movie-Dialogs Corpus (about 600 films' dialogue, with character names and genres), OpenSubtitles (dialogue), CMU Movie Summary Corpus (plot summaries and character names), and TMDB or IMDb (metadata).
- Baselines are clean. Random, untuned BM25, then your model.

One caveat. If you generate queries automatically (sample a line and perturb it), the labels are free but the queries are artificial. Mix in 30 to 50 real or hand-written queries, for example from Reddit or friends who describe movies from memory, so the evaluation holds up.

A good next question to settle is the document unit. Is a document a whole movie (all its dialogue concatenated), or a single scene or line that maps back to a movie? That choice changes the data and the metric.

Turn 4: Resume-to-job matching

> **You:** or how bout scraping the job listing from current available websites, and feed the resume so that the most matching jobs can be output?

Claude:

Yes, this counts, and it is a well-established IR task (resume-to-job matching). It's a stronger portfolio piece than the movie idea, since the PDF itself says the project should be something you can talk about in job interviews. The risks are in the data, not the idea.

Task framing

- Input (query): a resume. It's a long, semi-structured document, not a 3-word keyword query, which makes it a "query-by-document" task.
- Output: a ranked list of job postings.
- Relevance: how well the candidate fits the job, graded for example 0 = not relevant, 1 = partial fit, 2 = strong fit.

Main risks

Risk	Why it matters	Mitigation
Scraping	LinkedIn and Indeed forbid scraping in their ToS and block bots. The PDF warns against data that needs permissions or can fall through.	Use legitimate sources: Kaggle LinkedIn job-postings datasets, public APIs (Adzuna, USAJobs, RemoteOK), Greenhouse/Lever public job-board JSON, or the HN "Who is hiring" threads. Name one as a backup.
Resume privacy	Real resumes contain PII.	Use your own and teammates' resumes with consent, plus public resume datasets (Kaggle resume datasets) or synthetic ones.

No ground truth	Nobody has labeled which jobs match which resume.	You annotate. Take about 20 to 30 resumes and label the top 10 to 20 results pooled from your systems, which is roughly 400 to 600 judgments and doable for a team.
"Current" listings change	Results aren't reproducible if the data moves.	Freeze a dated snapshot.

What makes it more than BM25. BM25 with the whole resume as the query is your simple baseline, and it will do reasonably well. The gap to show could be:

1. Dense embeddings or cross-encoder reranking, which handle synonyms ("ML engineer" vs "data scientist").
2. Structured extraction. Pull skills, years of experience, seniority and location out of the resume, then use them as filters or features. Hard constraints like location, visa and seniority are exactly where pure text matching fails.
3. Skill-gap explanation. For each result, say which requirements are met and which are missing, then evaluate whether the explanation is correct.

Evaluation

- Metrics: nDCG@10 (graded relevance) and Precision@k.
- Baselines: random, untuned BM25, then your model.

Suggested scoping. To keep it manageable, restrict to one field (for example data and software roles, or one city), so annotation is consistent and the collection stays a reasonable size.

Compared with the movie idea, this one has the harder labeling cost (movie queries get labels free from the known target) but a much better real-world story. If you have the time to annotate, I'd pick this one. If you want less risk, the movie-quote idea is safer.

Turn 5: Mimicking a movie site; multimodal, global and multilingual

> **You:** https://67movies.st/ check the website and now we are thinking to mimic this website by having the search system even more broader, like searching not only the text, genre, but also the photos, images, descriptions, etc. also, the source is not restricted to movies, but also tv shows, animations across all regions (vpn includded maybe). Lastly, the query itself require the function that is able to support multi-language so that when people type chinese, chinese source may pop up first even though final result is english.

Claude: (fetched the site's home page)

I loaded the site. It presents itself as a free "watch movies and TV online" service. Its footer says it hosts no streams and links to content on third-party sites. There's no licensing, DMCA or rights-holder

information, and the excerpt I got didn't describe search or filters, so I can't tell you how its search works.

What I won't help with. Sites like this are typically unauthorized streaming aggregators. I wouldn't build the project around scraping it, linking to its streams, or using a VPN to reach region-locked streams. That's a copyright and ToS problem. It's also a practical risk for a graded project, because the PDF warns against data that can disappear or require permissions, and these sites go down constantly.

You don't need any of that. A search system only needs metadata: titles, descriptions, genres, cast, posters. Sources like TMDB (free API, global coverage, many languages, posters), Wikipedia/Wikidata, and the IMDb datasets are legitimate and stable. The "search engine" is the project, not the streaming.

Your three extensions

Extension	Verdict	Notes
Multi-language queries	Best core idea	This is cross-lingual IR (CLIR), a real research area. TMDB stores titles and overviews in many languages for the same item, so you get aligned data and free relevance labels. Baselines: untuned BM25, then query translation then BM25, then a multilingual dense model (e.g. multilingual-e5, BGE-M3).
Multimodal (posters, images)	Good as an extension	Text to poster or image to movie with CLIP-style models. Posters come from TMDB. Known-item queries give labels.
All regions and types (TV, anime, global)	Just scope	TMDB already covers it. It enlarges the collection but isn't a research contribution by itself.

Scope warning. Doing all three in one semester is too big. I'd make multilingual retrieval the core, and add images only if you're ahead at Week 11. Without that cut, the evaluation gets messy, and Evaluation is worth 30 points.

One thing to clarify. You wrote "when people type Chinese, Chinese source may pop up first even though final result is English." There are two different readings:

1. Cross-lingual retrieval: a Chinese query finds the right movie, even when the document is in English. Relevance is about content, not language.
2. Language-preferring ranking: Chinese-language or Chinese-origin titles get boosted when the query is in Chinese.

These need different metrics. Reading 1 is cleaner to evaluate (MRR, nDCG against a known target). Reading 2 needs a definition of "should be first", which is subjective. Which one do you mean?

Turn 6: What "core" means; dataset size and TMDB

> **You:** what do u mean by having multi-language as the core while others are not. how to set the dataset size? still from tmdb?

Claude:

What "core" means. In your proposal the project needs one clear research question that you measure rigorously. The other features can exist in the system, but you don't evaluate each of them.

Role	What it means	Example in your project
Core	The thing you define a task, metric and baselines for.	"Does a multilingual dense retriever beat translate-then-BM25 for Chinese/Spanish queries over movie metadata?"
Extension	Added if time allows. It gets a small demo or a modest evaluation, not the main one.	Poster/image search with CLIP.
Scope	Which items are in the collection. It costs nothing in research effort.	Movies, TV and anime from all regions.

So "multilingual as core" means the 30-point Evaluation section is about cross-lingual retrieval. The poster search might exist as a bonus feature, but if it's half-evaluated it can hurt more than help.

Dataset size and source. Yes, TMDB is still the right source. It's free, has an API, covers every region and type, and keeps titles and overviews in many languages for the same item.

How to size it:

- TMDB has well over a million titles, and most of them have no overview or no translations, so they're useless to you.
- Filter to items that have an English overview and a translation in each language you test. Then keep the top N by popularity or vote count.
- Target 10k to 30k items. That's big enough that retrieval is a real problem and baselines differ, and small enough to embed on a laptop or free Colab GPU in minutes. Don't go below about 5k, because the task gets too easy.

How to collect it:

1. Get the list of IDs (TMDB publishes daily ID exports, or use the discover endpoint).
2. Fetch each item's details, using append_to_response=translations to get all languages in one call.
3. Store a snapshot with the date.

For the proposal's rough statistics table, report items by type (movie/TV), by original language, by genre, and how many have translations in each test language.

Backup source (the PDF asks for one): Wikipedia/Wikidata, which has multilingual labels and descriptions for films, or a Kaggle TMDB dump.

Queries and relevance

- Pick 2 to 3 query languages, such as Chinese plus Spanish, and maybe Japanese or Korean.

- Index the English fields only (title, overview, genres). The query is in the other language and the relevant document is the item it came from. That gives you free labels.
- Synthetic queries: use a sentence from the translated overview, or the translated title, as the query. Cheap and gives hundreds of queries, but easier than real queries because translations track the original closely.
- Hand-written queries: add 30 to 50 natural queries per language written by you or teammates. These make the evaluation believable.
- Metrics: MRR and Success@k (is the right item in the top k), plus nDCG if you add graded labels.

Two small notes: TMDB's terms require attribution and non-commercial use, which is fine for a class project but worth one line in the Data section. Also keep a precise count of how many items are dropped by your filters, since that goes straight into the statistics table.

Turn 7: Multimodal as an ablation; is the scope enough

> **You:** ok, so multilingual is the core, how bout multimodal? scope is 10k to 30k plus multilingual is enough for this project scope?

Claude:

Yes, that scope is enough, and I'd keep multimodal as a single add-on experiment rather than a second project.

Is 10k to 30k plus multilingual enough? The collection size isn't what makes a project big enough. What matters is how many things you compare and analyze. Multilingual retrieval on its own can fill the Evaluation section if you do these:

- Systems: random, untuned BM25 on English fields, translate-the-query then BM25, and a multilingual dense retriever (optionally with a reranker).
- Breakdowns: results per query language, per query type (title, plot description, vague mood), and per item type (movie, TV, anime).
- Error analysis: where cross-lingual retrieval fails, such as transliterated names, local title differences and cultural references.

That is already a full semester of work for a small team.

Where multimodal fits. The cleanest way to include it is as an ablation of the core task, not a separate task:

> Does adding the poster as an extra signal improve retrieval for multilingual text queries?

	Design
Documents	Each item = English text fields + poster image (from TMDB).
Queries	The same multilingual queries you already have.

Systems	Text-only, then text + poster fusion (a CLIP-style multilingual model embeds the poster and the query, then you combine scores with the text score).
Metric	The same MRR/Success@k/nDCG, so the comparison is direct.
Cost	Posters at a small size (about 1 GB total for 30k) and an embedding pass on a free GPU.

The expected result is that posters help for visual or mood queries ("dark, rainy, noir city") and don't help for plot queries ("a man steals dreams"). Either outcome is a legitimate finding, and it ties back to your multilingual core, because the CLIP text encoder has to handle Chinese or Spanish too.

I'd avoid image-to-movie (upload a screenshot, find the title) as a second full task, since it needs its own queries, labels and baselines.

How to write it in the proposal. Use tiers, because the Work Plan is graded on being concrete:

1. Must have: multilingual retrieval with baselines and evaluation.
2. Should have: the poster-fusion ablation, in the second half of the schedule.
3. Stretch: image-as-query demo.

Set a decision point at Week 11, when you must have your data and a simple baseline anyway. If the core isn't done by then, drop the ablation.

How many people are on your team? With 2 to 3 people, the ablation is comfortable. Working alone, I'd demote it to stretch.

Turn 8: Two-person team; scope and split

> **You:** its 2 people project.

Claude:

With two people, the plan works as tiered. I'd tighten it slightly.

Scope for a 2-person team

- Collection: aim for about 10k to 20k items. This keeps the pipeline fast to rerun and the posters small.
- Query languages: two, such as Chinese plus one more. Every extra language adds hand-written queries and error analysis.
- Multimodal ablation: keep it as "should have". It's comfortable for two people only if the core is finished by Week 11. Otherwise drop it.

Suggested split

	Person A: data and evaluation	**Person B: models and queries**

Early	TMDB collection, filtering, snapshot, statistics table	Query design, synthetic query generation from translations
Middle	Evaluation harness (MRR, Success@k, nDCG), random and BM25 baselines	Translate-then-BM25 and the multilingual dense retriever
Later	Error analysis by language, query type and item type	Poster-fusion ablation (if on track)
Together	Write 30 to 50 hand-written queries per language, and write the report	

Two details to keep in mind:

- The proposal's Data section needs the statistics table, so Person A should have a first pass of the collection counts before you write it.
- Keep the evaluation harness separate from the models. Then every new system plugs in and is compared on identical queries.

Next step. You now have most of the pieces for the proposal (task, data, baselines, metrics, plan). Per the PDF's genAI rule, you should write the sections yourselves. I'm happy to help in ways that fit that policy:

- ask you questions to sharpen your task definition and example queries
- critique a draft you've written
- suggest where to search for the 5 related papers (for example CLIR benchmarks like Mr. TyDi/MIRACL, mDPR/BGE-M3, and CLIP-based retrieval)

Keep a copy of this conversation, since the PDF requires you to submit your genAI chat history.

> **You:** save as link so that other people can see such chat

Claude: Saved this conversation as a shareable doc (turns 1 to 8 above).

> **You:** pdf file pls

Claude: Exported this conversation as the PDF you are reading.